

MongoDB Schema Mastery: The Ultimate Viva Prep Dossier



Decoded trapdoors, examiner secrets, and the exact blueprints you need to survive the grilling. ?



*CRITICAL!
Must know this.*

Examiner's Note: Expect tough questions on indexing and schema design trade-offs. Be prepared.

SYSTEM
INITIALIZING...

The Gauntlet Roadmap



Level 1: Basic (Mental Models)

Start Here!
Foundation is key. Don't skip.



Why Schema?

Level 2: Interview Ready (The Fundamentals)



JSON Anatomy

MUST KNOW!
Core structure.

Practice writing valid JSON from scratch.

Level 3: Viva Ready (Complexity)



Nested & Arrays

Beware traps!
Check for mismatched types in arrays.

Increased difficulty.
Get comfortable with deep nesting.

EXAM ESSENTIAL!
This will break you.

Level 4: The Grilling (Advanced Trapdoors)

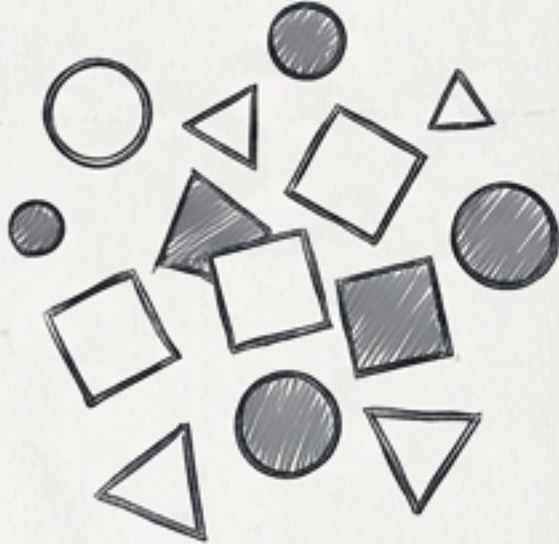


collMod & Bulk Inserts

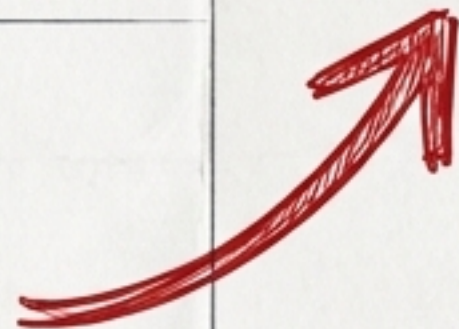
High failure rate here.
Master the edge cases & error handling!

EXAM ESSENTIAL!
This will break you.

The Database Philosophy: Schema vs. Schemaless

Schemaless (The Wild West)	Schema Validated (The Fortress)
 Agility: High Data Integrity: Low App-side logic handles all validation.	 Agility: Controlled Data Integrity: Bulletproof Database-side logic blocks buggy backend data.

*Examiners love this:
It prevents bad data
from a buggy backend!*



*Cheat Code #1:
Schema validation
is strict typing for
a NoSQL database.*

The Blueprint: Anatomy of \$jsonSchema

Block 1: The Wrapper

```
db.createCollection('users', { validator: ... })
```

Block 2: The Engine

```
$jsonSchema: { bsonType: 'object', ... }
```

Block 3: The Rules

```
required: ['name', 'age'], additionalProperties: false
```

Block 4: The Data Definition

```
properties: {  
  name: { bsonType: 'string', description: 'must be string' } }
```

Trapdoor: The
“description” field is
is purely for developer
comments. It will **NOT**
show up in MongoDB
error messages!

The Bouncer and the Sneaky _id



```
properties: {  
  _id: { bsonType: 'objectId' },  
  name: { ... }  
}
```

No extras allowed?
Then `_id` must be
avowed.
You **MUST** explicitly
define `_id` if
additional properties
are blocked.

Enforcing Stringency with Regex

The Password Vault

```
password: {  
  bsonType: 'string',  
  pattern: '^(?=.*[a-z])(?=.*[A-Z])(?=.*  
  *\\d)(?=.*[@$!%*?&])[A-Za-z\\d@!%*?&]{8,}$'
```

Min 8 chars

1 uppercase

1 lowercase

1 number

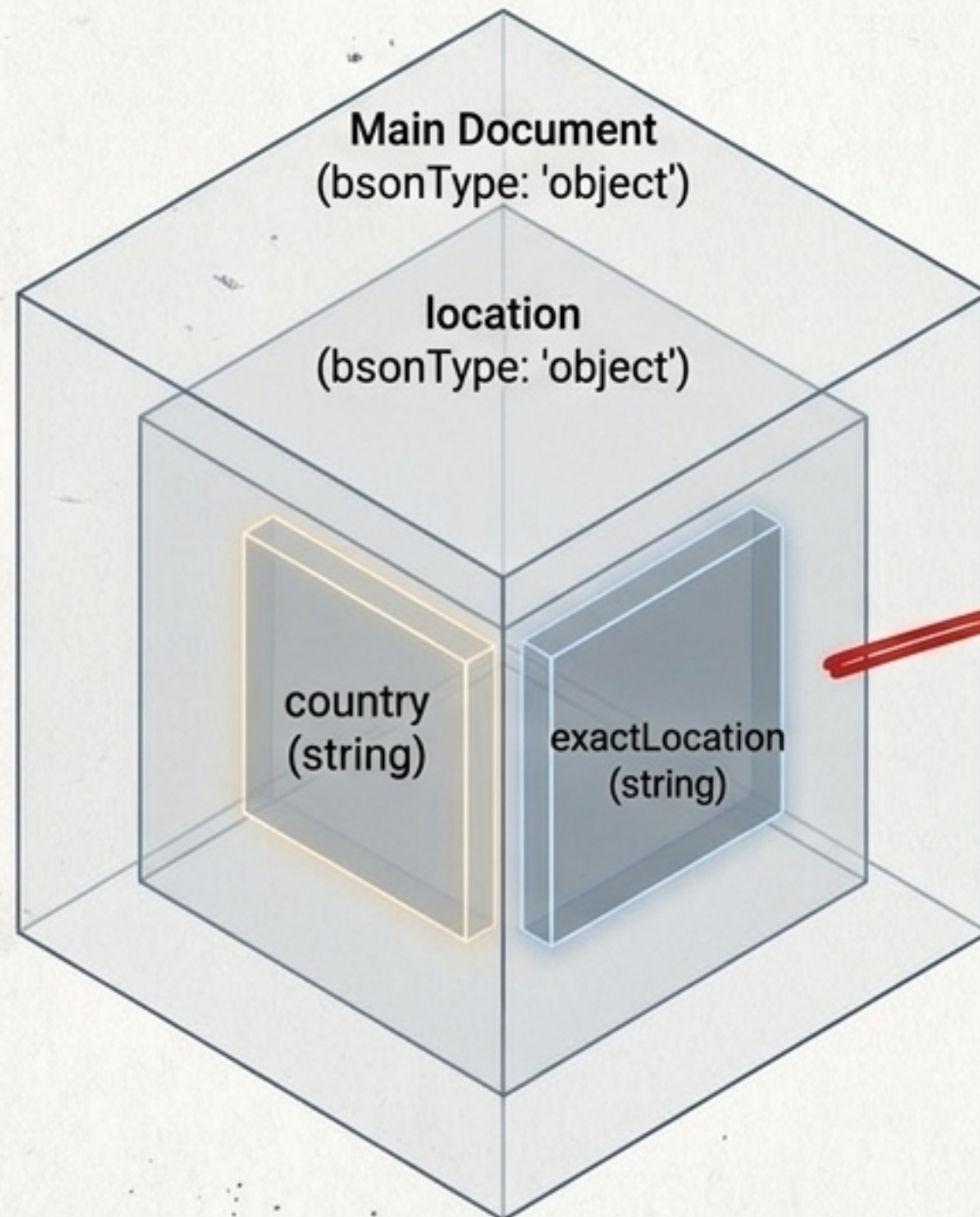
1 special char

The Email Gatekeeper

```
email: {  
  bsonType: 'string',  
  pattern: '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$'
```

Regex in Mongo requires double backslashes '\\d' in the string literal to escape properly!

The Nested Russian Doll



```
location: {  
  bsonType: 'object',  
  required: ['country', 'exactLocation'],  
  properties: {  
    country: { bsonType: 'string' },  
    exactLocation: { bsonType: 'string' }  
  }  
}
```

Arrays & The Items Rule

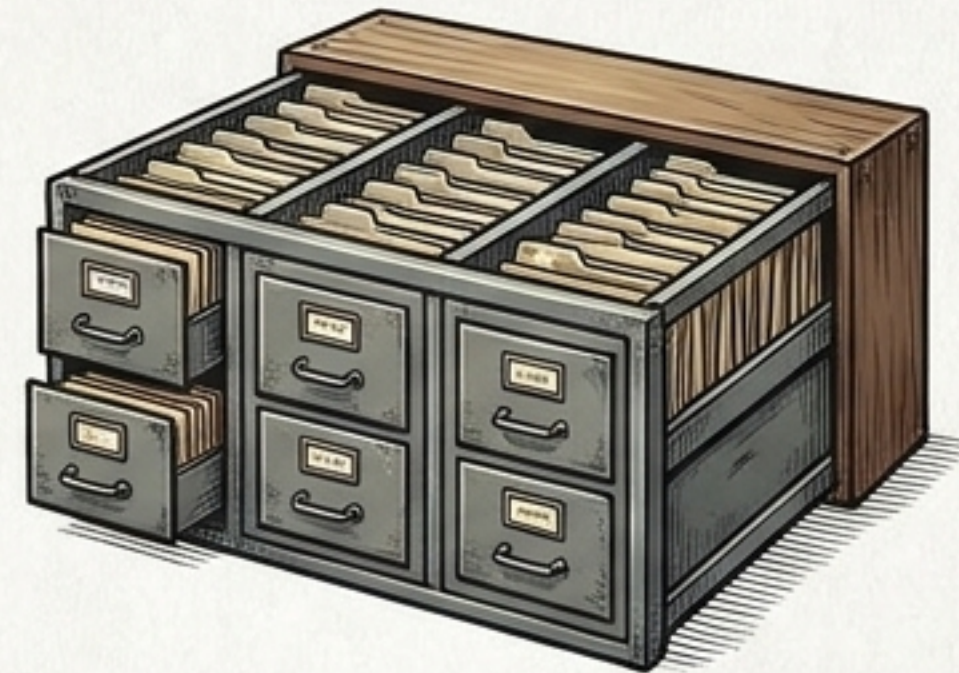
Simple Array (favTeacher)



```
bsonType: 'array',  
items: {  
  bsonType: 'string'  
}
```

The 'items' keyword is the recursion trigger—it acts as a mini-schema for everything inside the array!

Complex Array (favSubject)



```
bsonType: 'array',  
items: {  
  bsonType: 'object',  
  required: ['bookName', 'bookAuthor'],  
  properties: { ... }  
}
```

Retrospective Enforcement: The collMod Command

Existing Collection

```
db.createCollection('users', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['username'],
      properties: {
        username: {
          bsonType: 'string',
          description: 'must be a string'
        }
      }
    }
  }
})
```

Error: Collection already exists!

**X CANNOT
REDEFINE!**

MODify the
COLLecTion
= collMod.

Retrospective Application

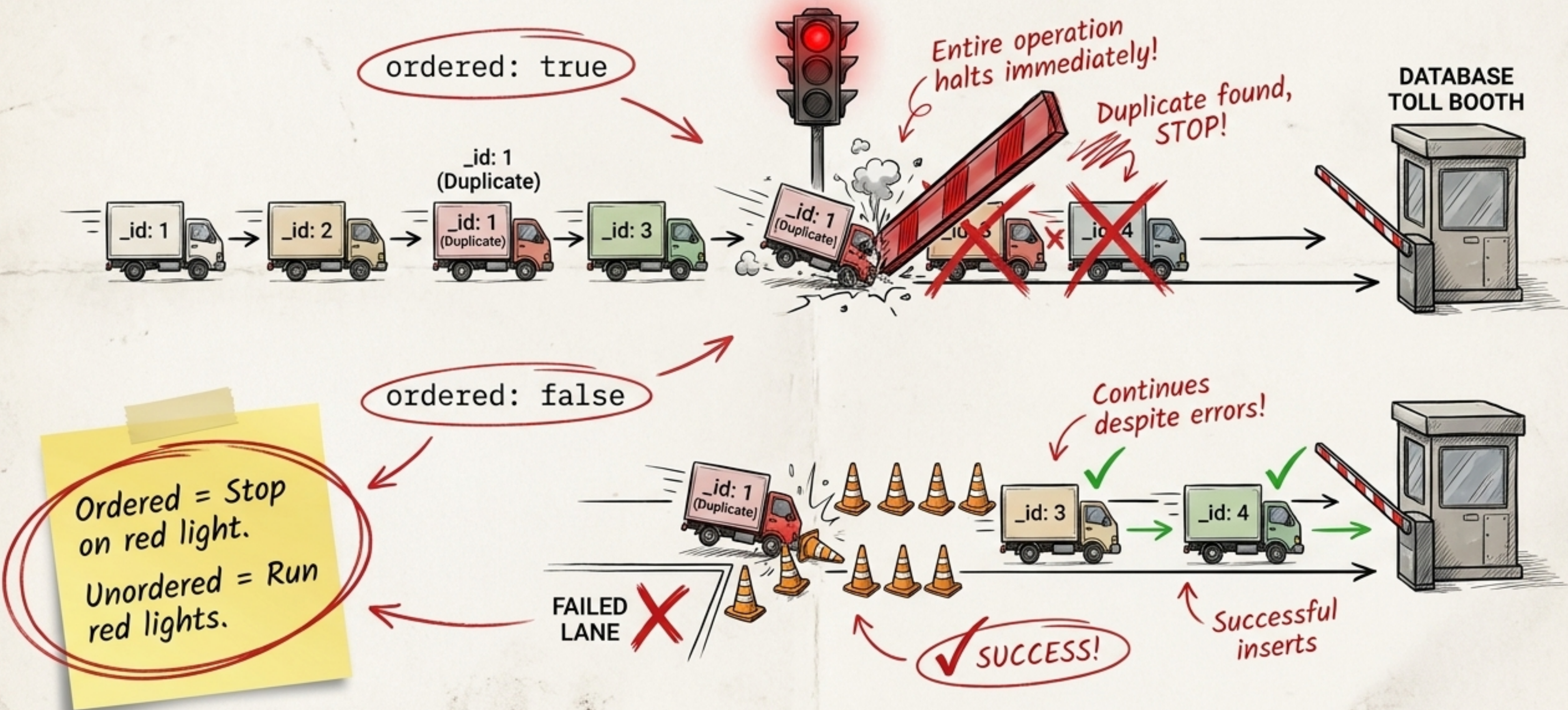
```
db.runCommand({
  collMod: 'users',
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['username', 'email'],
      properties: {
        username: {
          bsonType: 'string',
          minLength: 3,
          maxLength: 20,
          description: 'must be a string with min/max length'
        },
        email: {
          bsonType: 'string',
          pattern: '^.+@.+$',
          description: 'must be a valid email'
        }
      }
    }
  }
})
```

ok: 1, nModified: 0, ...

✓ SUCCESS!

Examiner Trap: collMod updates the rules, but it DOES NOT automatically delete existing bad data. It only blocks **new** bad inserts or updates!

Bulk Inserts: The Traffic Light Protocol



The Insert Matrix

Feature	ordered: true	ordered: false
Default Status	Yes	No (Must specify)
Primary Use Case	Strict sequential entry	Bulk log imports, skipping duplicates
Duplicate Key Error	Aborts entire operation	Skips duplicate, inserts the rest
Syntax Error	Aborts operation	Aborts operation

➔ **FATAL FLAW:** Even in unordered inserts, a syntax error kills the ENTIRE batch. It only bypasses constraint errors!

The Examiner's Rapid Fire

MAX LEVEL -
Exam Simulator

Q: Why do inserts fail when `additionalProperties: false` even if data looks perfect?

✓ A: Because MongoDB auto-generates `_id`. It must be explicitly allowed.

CRUCIAL!
Don't forget.

Q: What is the default behavior of `insertMany` if document #2 fails a unique index constraint?

✓ A: It defaults to `ordered: true`, aborting the operation. Docs #3 onwards are discarded.

Chain reaction failure!

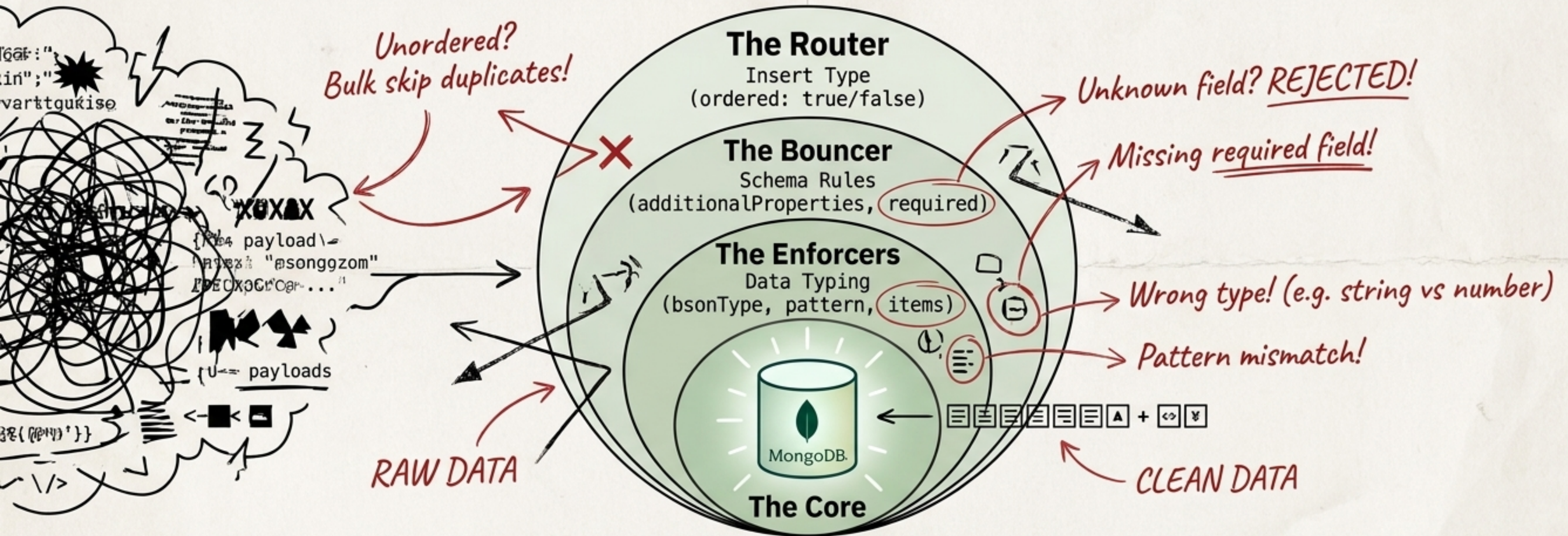
Q: I have a syntax error in my JSON array, but I used `ordered: false`. Will the valid docs save?

✓ A: No. Syntax errors fail the entire operation immediately, bypassing the unordered logic.

Syntax errors = instant death for the batch.

System Secured

Synthesis: The Data Defense Shield



MongoDB validation isn't just a syntax check; it is a multi-layered firewall engineered to protect data integrity at every stage of ingestion.

→ *CRITICAL CONCEPT - Think defense in depth!*

The Final Viva Checklist



- ☒ [x] Schema validation is **strict typing** for NoSQL.
- ☒ [x] `additionalProperties: false` requires an **explicit** `_id` rule.
- ☒ [x] Nested **objects** and **arrays** require their own **internal properties** or `items` schemas.
- ☒ [x] Use `collMod` to enforce rules on existing collections without dropping data.
- ☒ [x] `ordered: true` stops at the first error; `ordered: false` skips duplicates (but fails on syntax errors).

Dossier Compiled. Good luck in the Gauntlet.

